

CryptoTrack

System wizualizacji i analizy transakcji MetaMask Card

Artem Zakharov

Przedmiot: Systemowe Projektowanie

Rok akademicki: 2025/2026

1. Cel projektu

Celem projektu jest stworzenie funkcjonalnego oprogramowania umożliwiającego użytkownikom kart MetaMask Card śledzenie wydatków w sposób zbliżony do tradycyjnych aplikacji bankowych. Transakcje kryptowalutowe prezentowane w portfelu MetaMask są trudne do analizy pod kątem codziennych wydatków — brakuje kategoryzacji, podsumowań miesięcznych oraz przejrzystych wykresów.

CryptoTrack rozwiązuje ten problem poprzez:

- synchronizację historii transakcji z panelu MetaMask Card (rozszerzenie Chrome),
- przechowywanie i normalizację danych w centralnej bazie PostgreSQL,
- ręczną i automatyczną (AI) kategoryzację wydatków według kategorii i podkategorii,
- dashboard, listę transakcji oraz moduł analityki z wykresami i porównaniem okresów.

Ostatecznym efektem jest podniesienie świadomości finansowej użytkownika Web3 oraz ułatwienie kontroli budżetu opartego na wydatkach kartą.

2. Szczegółowy opis wymagań

2.1. Wymagania funkcjonalne

❖ Obsługa kont użytkowników

- Rejestracja i logowanie za pomocą adresu e-mail i hasła (JWT + sesje odświeżania).
- Opcjonalne logowanie portfelem Web3 (SIWE — Sign-In with Ethereum).
- Profil użytkownika: wyświetlana nazwa, powiązany adres portfela (PortfolioAccount).

❖ Synchronizacja transakcji MetaMask Card

- Rozszerzenie Chrome (Plasmo) pobiera listę transakcji ze stron `card.metamask.io` / `portfolio.metamask.io`.
- Parowanie rozszerzenia z kontem użytkownika kodem 6-cyfrowym wygenerowanym w ustawieniach aplikacji web.
- Endpoint REST `POST /api/v1/card-transactions/sync` — upsert transakcji po `externalId`.
- Zapisywane pola: `merchant`, kwota fiat/crypto, status (`PENDING`, `SETTLED`, `DECLINED`, `REFUNDED`), znacznik czasu.

❖ Zarządzanie transakcjami

- Lista transakcji z filtrowaniem po dacie, kategorii, merchantcie, statusie.
- Ręczna edycja kategorii, podkategorii i notatek dla pojedynczej transakcji.
- Statystyki zbiorcze: sumy wydatków, rozkład po kategoriach (`GET /transactions/stats`).

❖ Kategorie i podkategorie

- CRUD kategorii użytkownika (nazwa, kolor, ikona).
- Hierarchia `parent` → `subcategory` (np. „Jedzenie” → „Restauracje”).
- Kategorie systemowe (seed) oraz domyślne podkategorie.
- Strona „Categorize spendings” — masowa kategoryzacja merchantów.

❖ Automatyczna kategoryzacja (AI + pamięć merchantów)

- Model `cardMerchantMemory` — zapamiętanie mapowania `merchantKey` → kategoria po potwierdzeniu użytkownika lub AI.
- Usługa OpenAI przypisuje kategorię nieznanym merchantom (`bulk-categorize`).
- Śledzenie przebiegu w `CardCategorizationRun` (status, liczniki dopasowań).

❖ Analityka i wykresy

- Strona Analytics: wykresy wydatków w czasie, rozkład kategorii, porównanie okresów.
- Biblioteki: `Recharts`, agregacja po stronie API.
- Filtr zakresu dat (`DateRangePicker`).

❖ Dashboard

- Podsumowanie ostatnich transakcji, saldo karty (snapshot z rozszerzenia), szybki wgląd w wydatki.

2.2. Wymagania нефункционалне

❖ Łatwość użycia

- Responsywny interfejs (Next.js 15, React 19, Tailwind CSS, shadcn/ui).
- Sidebar nawigacyjny: Dashboard, Transactions, Categories, Settings; widok mobilny z menu.
- Tryb jasny/ciemny (ThemeSwitcher).

❖ Niezawodność

- Monorepo (pnpm, Turbo): spójne typy w pakiecie `@crypto-tracker/shared`.
- Walidacja DTO (class-validator) i obsługa błędów HTTP w NestJS.
- Testy jednostkowe usług API (Jest).

❖ Wydajność

- PostgreSQL z indeksami na `userId`, `timestamp`, `categoryId`.
- Paginacja list transakcji; agregacje statystyk po stronie serwera.

❖ Bezpieczeństwo

- Hasła hashowane (bcrypt); tokeny JWT krótkotrwałe + refresh w bazie (hash).
- Tokeny rozszerzenia przechowywane jako hash (`ExtensionToken`).
- CORS i guardy autoryzacji na endpointach chronionych.
- TLS w środowisku produkcyjnym (HTTPS).

3. Ograniczenia

3.1. Ograniczenia technologiczne

- Wymagane połączenie sieciowe do synchronizacji z API oraz (przy kategoryzacji AI) do OpenAI.
- Rozszerzenie działa tylko w przeglądarce Chrome/Chromium z uprawnieniami do domen MetaMask.
- Źródło danych ograniczone do transakcji MetaMask Card (nie pełna historia on-chain wszystkich sieci).

3.2. Ograniczenia zakresu

- Brak wieloużytkownikowych zespołów i ról menedżerskich (aplikacja single-user per konto).

- Brak integracji z systemami płatności fiat poza danymi z karty MetaMask.

3.3. Ograniczenia czasowe

- Projekt realizowany w ramach roku akademickiego 2025/2026.

4. Użytkownicy

4.1. Kluczowy użytkownik

❖ **Użytkownik karty MetaMask (właściciel konta)** — rejestruje się, paruje rozszerzenie, synchronizuje transakcje, zarządza kategoriami, przegląda analitykę i edytuje przypisania.

4.2. Użytkownicy techniczni

❖ **Administrator wdrożenia** — utrzymuje PostgreSQL, zmienne środowiskowe (DATABASE_URL, JWT_SECRET, OPENAI_API_KEY), deployment Docker.

5. Nazewnictwo i definicje

- Użytkownik** — osoba posiadająca konto w systemie CryptoTrack.
- Transakcja karty** — operacja zarejestrowana na koncie MetaMask Card (merchant, kwota, status).
- Merchant** — nazwa punktu handlowego (znormalizowany klucz `merchantKey`).
- Kategoria / podkategoria** — etykieta klasyfikacji wydatku (hierarchia drzewiasta).
- Pamięć merchantów (CardMerchantMemory)** — trwałe mapowanie merchant → kategoria.
- Rozszerzenie Chrome** — moduł Plasmo scrapujący HTML listy transakcji MetaMask.
- SIWE** — Sign-In with Ethereum; logowanie podpisem portfela.
- JWT** — JSON Web Token używany do autoryzacji żądań API.

6. Diagram przypadków użycia (UML)

[WSTAW SCREENSHOT]

Eksport PNG z pliku:

docs/University/diagrams/cryptotrack-use-case.drawio

Otwórz w <https://app.diagrams.net> → File → Export as → PNG

Diagram obejmuje aktorów: Użytkownik, Rozszerzenie Chrome oraz przypadki: rejestracja, SIWE, synchronizacja, przeglądanie transakcji, kategorie, kategoryzacja AI, analityka, parowanie rozszerzenia.

7. Scenariusze przypadków użycia

7.1. Rejestracja nowego użytkownika

Pole	Wartość
Nazwa przypadku użycia	Rejestracja nowego użytkownika
Cel	Utworzenie konta w systemie CryptoTrack
Warunki wstępne	Użytkownik nie jest zalogowany; e-mail nie jest zajęty
Warunek pomyślnego zakończenia	Konto utworzone, możliwe logowanie
Stan końcowy — niepowodzenie	Komunikat błędu (np. zajęty e-mail, słabe hasło)
Główni aktorzy	Użytkownik
Wywołanie	Wejście na /auth/login →

Główny przepływ:

1. Użytkownik otwiera formularz rejestracji.
2. Wprowadza e-mail, hasło, opcjonalnie nazwę wyświetlaną.
3. System waliduje dane i zapisuje użytkownika w tabeli `users`.
4. Użytkownik zostaje przekierowany do logowania lub automatycznie otrzymuje tokeny.

7.2. Parowanie rozszerzenia Chrome

Pole	Wartość
Nazwa przypadku użycia	Parowanie rozszerzenia
Cel	Powiązanie rozszerzenia z kontem użytkownika
Warunki wstępne	Użytkownik zalogowany w aplikacji web
Warunek pomyślnego zakończenia	Token rozszerzenia zapisany w <code>chrome.storage</code>

Główny przepływ:

1. W Settings użytkownik generuje 6-cyfrowy kod parowania.
2. W popup rozszerzenia wpisuje kod i klika Connect.
3. API weryfikuje kod (`POST /auth/extension/pair`) i zwraca bearer token.

7.3. Synchronizacja transakcji

Pole	Wartość
Nazwa przypadku użycia	Synchronizacja transakcji karty
Warunki wstępne	Rozszerzenie sparowane; otwarta karta MetaMask w przeglądarce
Warunek	Transakcje widoczne w <code>/transactions</code>

pomyślnego
zakończenia

1. Użytkownik otwiera card.metamask.io i klika „Sync now” w rozszerzeniu.
2. Content script scrapuje wiersze tabeli transakcji.
3. Background normalizuje dane i wysyła POST sync z tokenem.
4. API wykonuje upsert w tabeli `transactions`.

7.4. Automatyczna kategoryzacja wydatków

1. Użytkownik wchodzi w Categories → Categorize spendings.
2. System skanuje nieprzypisane merchanty; najpierw dopasowuje z CardMerchantMemory.
3. Pozostałe merchanty kierowane do OpenAI; wynik zapisywany i uczy pamięci.
4. Użytkownik widzi postęp w CardCategorizationRun.

7.5. Przeglądanie analityki

1. Użytkownik otwiera /analytics.
2. Wybiera zakres dat.
3. API zwraca agregaty; frontend renderuje wykresy (Recharts).

8. Diagramy czynności (UML)

[WSTAW SCREENSHOT 8.1]

Diagram czynności: Rejestracja użytkownika

(narysuj w draw.io: Start → Formularz → Walidacja → [błąd|zapis DB] → Koniec)

[WSTAW SCREENSHOT 8.2]

Diagram czynności: Parowanie rozszerzenia

[WSTAW SCREENSHOT 8.3]

Diagram czynności: Synchronizacja transakcji

[WSTAW SCREENSHOT 8.4]

Diagram czynności: Automatyczna kategoryzacja (AI)

9. Diagram ERD

[WSTAW SCREENSHOT]

Eksport z: docs/University/diagrams/cryptotrack-erd.drawio

10. Diagram klas (UML)

[WSTAW SCREENSHOT]

Klasy domenowe: User, Transaction, Category, CardMerchantMemory,

CardCategorizationRun, ExtensionToken, PortfolioAccount — powiązania 1:N i FK jak w

Prisma schema

11. Diagram sekwencji (UML)

[WSTAW SCREENSHOT]

Sekwencja: Użytkownik → Rozszerzenie → API → PostgreSQL przy synchronizacji (Sync

now)

12. Diagram stanów (UML)

[WSTAW SCREENSHOT]

Stany transakcji karty: PENDING → SETTLED | DECLINED | REFUNDED

[WSTAW SCREENSHOT]

Stany CardCategorizationRun: QUEUED → RUNNING → COMPLETED | FAILED |

PARTIAL

13. Diagram komponentów i wdrożenia (UML)

[WSTAW SCREENSHOT]

Eksport z: docs/University/diagrams/cryptotrack-deployment.drawio

14. Plan procesu testowania

14.1. Testy automatyczne

- **Testy jednostkowe** — Jest: `card-transactions.service.spec.ts`, `card-merchant-auto-categorize.spec.ts`.
- **Testy integracyjne** — (planowane) endpointy auth i sync z testową bazą.

14.2. Testy manualne

Scenariusze opisane w rozdziale 19 — wykonywane przez testera na środowisku dev (web + API + rozszerzenie).

14.3. Utrzymanie

Po wdrożeniu: raportowanie błędów, poprawki w kolejnych wersjach, migracje Prisma.

15. Raport z analizy rynku

15.1. Funkcje kluczowe CryptoTrack

- Specjalizacja w MetaMask Card (nie ogólny eksplorator blockchain).
- Kategoryzacja wydatków z AI i pamięcią merchantów.
- Rozszerzenie do synchronizacji bez ręcznego importu CSV.

15.2. Porównanie z konkurencją

❖ Koinly / CoinTracker

- Zalety: szerokie wsparcie podatków i wielu giełd.
- Wady: mniej nastawione na codzienne wydatki kartą DeFi; wyższy próg wejścia.

❖ Rotki

- Zalety: open source, lokalna kontrola danych.
- Wady: wymaga ręcznej konfiguracji; brak dedykowanego flow MetaMask Card.

❖ Aplikacje bankowe tradycyjne

- Zalety: gotowa kategoryzacja i UX.
- Wady: nie obejmują wydatków kartą krypto MetaMask.

15.3. Podsumowanie

CryptoTrack wypełnia niszę „expense tracker dla MetaMask Card”. Wybór narzędzia zależy od tego, czy użytkownik potrzebuje raportów podatkowych on-chain, czy przejrzystego budżetu kartowej.

16. Struktura bazy danych

16.1. Model koncepcyjny



[WSTAW SCREENSHOT]

Ten sam co diagram ERD (rozdz. 9) lub uproszczony model koncepcyjny bez typów SQL

16.2. Model logiczny

Encje: User (1) — (N) Transaction, Category, Session, ExtensionToken, CardMerchantMemory, CardCategorizationRun; User (1) — (1) PortfolioAccount; Category (1) — (N) SubCategory (self-reference parentId).

16.3. Model fizyczny

PostgreSQL; ORM Prisma; tabele: users, sessions, transactions, categories, card_merchant_memory, card_categorization_runs, extension_tokens, portfolio_accounts.

17. Dokumentacja bezpieczeństwa

17.1. Szyfrowanie w tranzycie

HTTPS (TLS 1.2+) między przeglądarką a API w produkcji.

17.2. Szyfrowanie w spoczynku

Hasła: bcrypt hash. Tokeny rozszerzenia i refresh: przechowywane jako hash w bazie. Szyfrowanie dysku — zależne od dostawcy hostingu PostgreSQL.

17.3. Uwierzytelnianie

JWT access + refresh token w tabeli sessions; opcjonalnie SIWE dla portfela Ethereum.

17.4. Ochrona przed atakami

- Walidacja wejścia (DTO, class-validator) — ochrona przed injection.
- Prisma parameterized queries — ochrona przed SQL injection.
- Rate limiting (planowane na produkcji).

17.5. Uprawnienia

Model single-tenant: każdy użytkownik widzi wyłącznie swoje transakcje (userId w każdym zapytaniu).

18. Kod SQL (PostgreSQL)

Poniżej fragment schematu wygenerowanego przez Prisma Migrate (dialekt PostgreSQL).

```
CREATE TABLE "users" (  
  
  "id" TEXT NOT NULL,  
  
  "email" TEXT NOT NULL,  
  
  "passwordHash" TEXT,  
  
  "displayName" TEXT,  
  
  "createdAt" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  
  "updatedAt" TIMESTAMP(3) NOT NULL,  
  
  CONSTRAINT "users_pkey" PRIMARY KEY ("id")  
  
);
```

```
CREATE UNIQUE INDEX "users_email_key" ON "users"("email");
```

```
CREATE TABLE "transactions" (
```

```
    "id" TEXT NOT NULL,
```

```
    "userId" TEXT NOT NULL,
```

```
    "source" "TransactionSource" NOT NULL DEFAULT 'CARD',
```

```
    "externalId" TEXT NOT NULL,
```

```
    "timestamp" TIMESTAMP(3) NOT NULL,
```

```
    "merchantName" TEXT,
```

```
    "fiatAmount" DECIMAL(18,2),
```

```
    "categoryId" TEXT,
```

```
    "subcategoryId" TEXT,
```

```

        CONSTRAINT "transactions_pkey" PRIMARY KEY ("id"),

        CONSTRAINT "transactions_userId_fkey" FOREIGN KEY ("userId")
REFERENCES "users"("id") ON DELETE CASCADE

);

CREATE UNIQUE INDEX "transactions_userId_source_externalId_key"

ON "transactions"("userId", "source", "externalId");

```

Pełny schemat: packages/db/prisma/schema.prisma oraz katalog prisma/migrations/.

18.2. Widok przykładowy (PostgreSQL)

```

CREATE VIEW v_user_spending_by_category AS

SELECT

    t."userId",

    c.name AS category_name,

    DATE_TRUNC('month', t."timestamp") AS month,

```

```

SUM(t."fiatAmount") AS total_fiat

FROM transactions t

LEFT JOIN categories c ON c.id = t."categoryId"

WHERE t."fiatAmount" IS NOT NULL

GROUP BY t."userId", c.name, DATE_TRUNC('month', t."timestamp");

```

19. Przypadki testowe

19.1. Rejestracja — niespójne hasło

Nr	Krok		Rezultat
1	Otwórz /auth/login → rejestracja		Formularz rejestracji
2	Wpisz różne hasła w polach Hasło i Powtórz		—
3	Kliknij Zarejestruj		Komunikat błędu walidacji; brak konta

19.2. Rejestracja — pomyślna

Nr	Krok		Rezultat
1	Poprawne dane, unikalny e-mail		—

2

3

Zarejestruj

Przekierowanie /
możliwość logowania

19.3. Synchronizacja bez parowania

N
r

Krok

Rezultat

1

Sync now
bez
tokenu

Komunikat błędu w popup
rozszerzenia

20. Testy jednostkowe

[WSTAW SCREENSHOT]

Wynik: `pnpm --filter @crypto-tracker/api test` — zielone testy Jest

Testy obejmują m.in. normalizację merchantów i logikę auto-kategoryzacji (`card-merchant-auto-categorize.spec.ts`).

21. Podręcznik użytkownika

Spis treści: 1. Rejestracja. 2. Logowanie. 3. Parowanie rozszerzenia. 4. Synchronizacja. 5. Transakcje. 6. Kategorie. 7. Kategoryzacja AI. 8. Analityka. 9. Ustawienia.

21.1. Rejestracja i logowanie

[WSTAW SCREENSHOT]

Strona: /auth/login — formularz logowania i link rejestracji

21.2. Dashboard

[WSTAW SCREENSHOT]

Strona: /dashboard — podsumowanie, ostatnie transakcje, saldo karty

21.3. Parowanie rozszerzenia

[WSTAW SCREENSHOT 1]

Web: /settings — sekcja Extension pairing, wygenerowany kod 6-cyfrowy

[WSTAW SCREENSHOT 2]

Chrome: popup rozszerzenia CryptoTrack — pole kodu, przycisk Connect

21.4. Synchronizacja transakcji

[WSTAW SCREENSHOT 1]

Karta: card.metamask.io — lista transakcji (zakładka Card)

[WSTAW SCREENSHOT 2]

Popup rozszerzenia — przycisk „Sync now”, komunikat sukcesu

21.5. Lista transakcji

[WSTAW SCREENSHOT]

Strona: </transactions> — tabela z merchantami, kwotami, badge kategorii, filtry dat

21.6. Zarządzanie kategoriami

[WSTAW SCREENSHOT]

Strona: </categories> — drzewo kategorii, dodawanie/edycja, kolory

21.7. Masowa kategoryzacja

[WSTAW SCREENSHOT]

Strona: */categories/categorize* — lista merchantów, przycisk auto-categorize, postęp

21.8. Analityka

[WSTAW SCREENSHOT]

Strona: */analytics* — wykresy wydatków, porównanie okresów, wybór zakresu dat

21.9. Edycja transakcji (kategoria)

[WSTAW SCREENSHOT]

Strona: */transactions* — otwarty dialog/sheet edycji pojedynczej transakcji z wyborem

kategorii

Stos technologiczny (podsumowanie)

TypeScript, React 19, Next.js 15, Tailwind CSS, NestJS, Fastify, PostgreSQL, Prisma, JWT, SIWE, REST API, rozszerzenie Chrome (Plasmo), OpenAI API, Docker, pnpm, Turbo.

Repozytorium

GitHub: [wstaw link do repozytorium]